

# Refinements on an enumeration scheme for solving a pattern sequencing problem

H. H. Yanasse and M. S. Limeira

*Laboratory for Computing and Applied Mathematics, Brazilian Space Research Institute*

Received 20 April 2003; received in revised form 9 September 2003; accepted 28 November 2003

---

## Abstract

We introduce some refinements on a branch- and bound-scheme for solving the minimization of open stack problem (MOSP). After representing the MOSP as a graph traversing problem, we attempt to divide the graph into parts aiming to solve the resulting subgraphs independently in order to reduce the search in the branching scheme. Subgraphs with special topologies (such as trees) are solved exactly using polynomial time algorithms. The branching scheme is applied only to the parts that are ‘complex’. The refinements introduced produce substantial savings in computational time when the MOSP graph presents some special structures. Limited computational results are presented.

*Keywords:* sequencing problem; open orders minimization; branch-and-bound; gate matrix layout; programmable logic array folding.

---

## Introduction

The pattern sequencing problem considered in this work is the minimization of open stack problem (MOSP) (see Yanasse, 1997b), that appears, for instance, in the context of cutting wood where there is limited storage space around the saw machine and stacks of panels being cut are to be removed only after their orders are completed. If the number of stacks increases too much, some of them have to be temporarily removed to another storage place and brought back afterwards to have their order completed. A stack is considered open when the first panel of that type is cut; a stack is considered closed only after all the orders of that panel type are completely cut. The MOSP consists in finding a sequence in which the patterns have to be cut in order to minimize the maximum number of open stacks during the production run. Consider, for instance, the set of cutting patterns shown in Table 1, where the panel types contained in each pattern are shown.

Observe that the number of times that a panel type is cut in each pattern is not indicated, since this information is not relevant for the MOSP, as only different panel types open new stacks.

Table 1

Set of cutting patterns.

| Patterns | Panels  |
|----------|---------|
| $P_1$    | a b d   |
| $P_2$    | b d e f |
| $P_3$    | c d     |
| $P_4$    | a c e   |
| $P_5$    | a d e   |
| $P_6$    | e f     |

Table 2

Stacks observed following sequence.

| Pattern sequenced | Open panels | Closed panels    | Number of open stacks |
|-------------------|-------------|------------------|-----------------------|
| $P_1$             | a b d       |                  | 3                     |
| $P_2$             | a b d e f   | Finished b       | 5                     |
| $P_3$             | a c d e f   |                  | 5                     |
| $P_4$             | a c d e f   | Finished c       | 5                     |
| $P_5$             | a d e f     | Finished a and d | 4                     |
| $P_6$             | e f         | Finished e and f | 2                     |

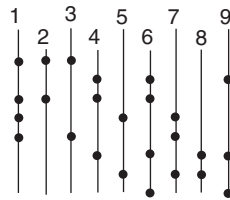


Fig. 1. A set of gates and transistors of a gate matrix layout style circuit. Reproduced from Linhares and Yanasse, 2002.

When we cut pattern  $P_1$ , for instance, three stacks will be opened, one for panel type  $a$ , one for panel type  $b$  and another for panel type  $d$ . The same number of stacks would be open if we had a pattern with three panels type  $a$ , four panels type  $b$ , and two panels type  $d$ . Notice also that to close a stack all patterns containing the corresponding panel type must be cut. Hence, say stack is a closed, only after patterns  $P_1$ ,  $P_4$ , and  $P_5$  are cut.

A possible sequence of the patterns is  $P_1$ ,  $P_2$ ,  $P_3$ ,  $P_4$ ,  $P_5$ ,  $P_6$ . This would result in the situation described in Table 2, with a maximum of five open stacks. Another possible sequence (an optimal one for this instance of the MOSP) is  $P_3$ ,  $P_4$ ,  $P_5$ ,  $P_1$ ,  $P_2$ ,  $P_6$  with a maximum of four open stacks.

MOSP also arises on VLSI design, and is known as the gate matrix layout problem (GMLP) (see Kashiwabara and Fujisawa, 1979; Linhares and Yanasse, 2002; Möhring, 1990; Ohtsuki, Mori, Kuh, Kashiwaba and Fujisawa 1979; Wing, Huang and Wang, 1985). A gate matrix layout circuit consists of a set of gates (vertical wires) with transistors (dots) that are used to interconnect the gates. Figure 1 shows such a set of gates before their interconnection.

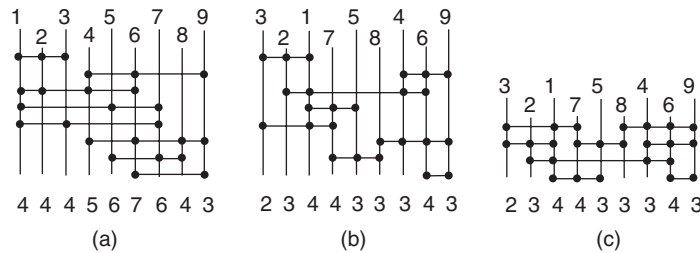


Fig. 2. Illustration of the *gate matrix layout problem* (GMLP). Reproduced from Linhares and Yanasse, 2002.

There must be horizontal wires known as *nets* interconnecting all the gates that share transistors at the same position. Non-overlapping nets can share the same *track*. The objective is to minimize the overall circuit area that reduces to minimizing the number of tracks for the number of gates is constant. The number of tracks is dependent upon the order in which the gates are sequenced, as illustrated in Fig. 2. The gate numbering is shown on the top of the figures. In Figs 2(a) and 2(b) distinct gate permutations are displayed, with the corresponding variation of nets at each column shown at the bottom of the figures. The permutation displayed in Fig. 2(b) yields a maximum column sum of 4, hence, it is possible to pack the layout in four tracks, as shown in Fig. 2(c).

The same ideas for GMLP apply to the Programmable Logic Array (PLA) folding problem, as pointed out in Ferreira and Song (1992). These problems are, hence, solution wise equivalent. Following these authors, PLA is a regular layout of logical gates that is commonly used to implement multiple-output combinatorial logic circuits, and PLA folding is a technique that attempts to produce an equivalent folded PLA that occupies less space.

The interested reader may refer to Linhares and Yanasse (2002), where a list of problems on VLSI design and in graph theory equivalent to the MOSP is presented. In Yanasse (1996, 1997b) there are examples of other productive systems where this same problem arises, its relation to other sequencing problems like the bandwidth minimization problem (see Garey, Graham, Johnson and Kuuth, 1978) or the minimization of order discontinuities problem (see Madsen, 1988), a mathematical formulation based on the minimization of the tool switches problem in a flexible manufacturing machine (see Tang and Denardo, 1988), a branch-and-bound algorithm and a heuristic procedure (see also Faggioli and Bentivoglio, 1998; Yuen, 1991) to solve general instances of the problem are presented. Additional reasons to motivate the study of this problem are also pointed out. In Yanasse (1996, 1997a) and Yanasse, Becceneri and Soma (1998), some polynomial time algorithms for solving special cases of MOSP are also presented. The general case of the problem is NP-hard (see, for instance, Linhares and Yanasse, 2002).

In Yanasse (1997b), a branch-and-bound scheme to solve the MOSP was proposed which uses an enumeration by panel type completion, that is, the optimal sequence is determined by identifying the order in which the panel types are completed. In Yanasse (1997c), any instance of MOSP was placed as a graph traversing problem, extending the result of Lins (1989, who introduced this approach for the special case where there are at most two panels per pattern).

By considering the MOSP as a graph traversing problem we can identify special structures of the problem that may facilitate its solution. In this work we propose refinements on the branch-and-bound scheme to solve the MOSP suggested in Yanasse (1997b) based on special structures

that may be eventually identified in the MOSP graph. To the best of our knowledge, such care has not been taken into account in any previous schemes proposed to solve the MOSP. Previously proposed schemes suggested in the literature use the same enumeration routines, regardless of whether the instance has some special structure.

With the refined algorithm, whenever possible, the MOSP graph is divided into parts, and each one of these parts is solved independently. The solutions of the parts are later combined to produce an optimal solution to the original problem. By dividing the graph into parts we reduce the number of enumerated nodes in the branch-and-bound scheme, hence consuming less computational time. The enumeration scheme is applied only to the parts of the graph that are ‘complex’; the parts of the graph that are ‘easy’ are solved using a simplified algorithm (sometimes a polynomial one).

In the following section, we present the basic result (Proposition 4) that supports the refinements suggested. In the third section we comment on how these results can be used to implement a refined algorithm for MOSP and, in the fourth section, we present some computational results comparing the new implemented algorithm with the previous one in Yanasse (1997b). Finally, we present some concluding remarks.

### Properties of an optimal solution to the MOSP

In order to make this text self-contained and facilitate its understanding, we reproduce some results here that are presented in Yanasse (1997c).

*Proposition 1* (from Yanasse, 1997c): Consider a sequence  $P_1, P_2 \dots P_n$  corresponding to a possible sequencing of the patterns of a MOSP. Let  $L$  be the list of open stacks following this sequence. The reverse order sequence of patterns  $P_n \dots P_2, P_1$  produces the same open stacks in reverse order, that is, list  $L$  in reverse order.

Proposition 1 assures that when we reverse a sequence, we obtain an alternative sequence with the same solution value.

*Proposition 2* (from Yanasse, 1997c): Suppose we have a MOSP,  $\text{PROB}_1$ , with  $N$  patterns to be sequenced. Suppose we have a second MOSP,  $\text{PROB}_2$ , with the same  $N$  patterns as in problem  $\text{PROB}_1$  plus a few additional patterns. Then the optimal solution value of  $\text{PROB}_2$  is greater than or equal to the optimal solution value of  $\text{PROB}_1$ .

Proposition 2 allows us to obtain lower bounds for a MOSP by determining a solution of smaller-sized, simpler MOSPs corresponding to subproblems (subset of patterns) of the original problem. Another trivial lower bound for the MOSP is the number of different panel types in a pattern.

*Proposition 3* (from Yanasse, 1997c): An optimal solution for any MOSP can be obtained from an optimal solution of a surrogate MOSP, appropriately built from the data of the original problem, where each pattern contains at most two panel types.

Using Proposition 3, any MOSP can be put as a graph traversing problem. When all patterns contain at most two panel types, we can build a graph associating nodes to panel types and arcs to

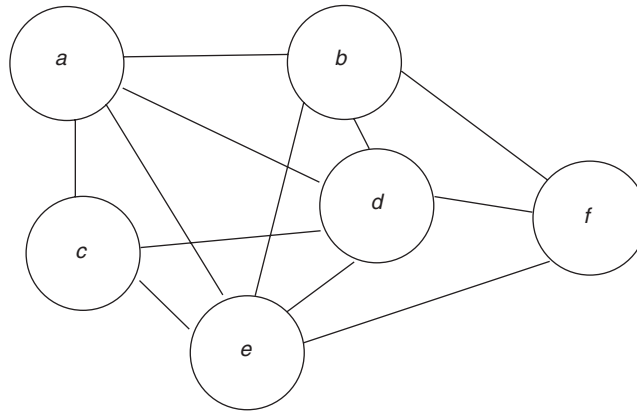


Fig. 3. MOSP graph corresponding to instance given in Table 1.

patterns (see Lins, 1989; Yanasse, 1997c). In this graph, which we will refer to as MOSP graph, the problem consists in finding an order in which all the arcs of the graph are to be traversed. Any permutation of the arcs of the MOSP graph defines an order in which the arcs can be traversed. If we denote a labeled node to be a node incident to some arc being traversed or already traversed and a completed node to be a node where all arcs incident to it have already been traversed, then the permutation we are interested in finding is one that minimizes the maximum number of labeled but not completed nodes while traversing the arcs. A solution to MOSP is simply a sequence of all the arcs of the graph.

The MOSP graph of a general instance of the problem is obtained as follows. For each panel type we associate a node in the corresponding MOSP graph. There is a one-to-one correspondence between nodes and panel types; hence, for simplicity of presentation, we will refer to nodes and panel types without distinction. In the MOSP graph, there is an arc connecting two nodes if and only if there is at least one pattern in the instance that contains both panel types. Observe that the MOSP graph is similar to the graph obtained by the translation of the GMLP to the graph problems path-width, or vertex separation number (see Kinnersley, 1992) that are equivalent problems to MOSP (see Linhares and Yanasse, 2002).

Consider the instance of MOSP given in Table 1. The corresponding MOSP graph (see Fig. 3) would have six nodes,  $a, b, c, d, e, f$ , one for each panel type, and all pairs of nodes are adjacent except for  $b$  and  $c$ ,  $c$  and  $f$ , and  $a$  and  $f$  since there are no patterns in this instance that contains panel types  $b$  and  $c$ ,  $c$  and  $f$ , or  $a$  and  $f$ .

A solution to the MOSP graph of Fig. 3 could be the following sequence of arcs  $\{(d, e), (b, d), (d, f), (b, f), (e, f), (b, e), (a, b), (a, d), (a, e), (c, d), (a, c), (c, e)\}$ . In Table 3 we present the labeled and completed nodes that are generated in correspondence to this arc traverse sequence. The resulting number of open stacks is also indicated.

The associated sequence of patterns in correspondence to this sequence of arcs can be obtained as indicated in Yanasse (1997c). The basic idea is the following: ‘follow the sequence of the arcs and sequence a pattern  $P_i$  when, for the first time, all the nodes corresponding to all the piece types in  $P_i$  are labeled’. For the sequence of arcs obtained, a possible pattern sequence solution is  $P_6, P_2, P_1, P_5, P_3, P_4$ .

Table 3

Labeled and completed nodes following sequence.

| Arc being traversed | Labeled nodes  | Completed nodes                      | Number of open stacks |
|---------------------|----------------|--------------------------------------|-----------------------|
| (d, e)              | <i>d e</i>     |                                      | 2                     |
| (b, d)              | <i>b d e</i>   |                                      | 3                     |
| (d, f)              | <i>b d e f</i> |                                      | 4                     |
| (b, f)              | <i>b d e f</i> |                                      | 4                     |
| (e, f)              | <i>b d e f</i> |                                      | 4                     |
| (b, e)              | <i>b d e</i>   | <i>f</i>                             | 3                     |
| (a, b)              | <i>a b d e</i> | <i>f</i>                             | 4                     |
| (a, d)              | <i>a b d e</i> | <i>f</i>                             | 4                     |
| (a, e)              | <i>a d e</i>   | <i>b f</i>                           | 3                     |
| (c, d)              | <i>a c d e</i> | <i>b f</i>                           | 4                     |
| (a, c)              | <i>a c e</i>   | <i>b d f</i>                         | 3                     |
| (c, e)              | <i>c e</i>     | <i>a b d f</i><br><i>a b c d e f</i> | 2                     |

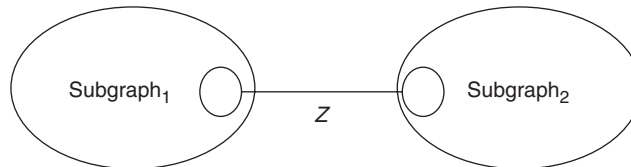


Fig. 4. Graph corresponding to a special instance of the MOSP.

From the literature we have been following, we have observed that there are many theoretical works related to MOSP or its equivalent problems; we can find some heuristics proposed to solve it, but exact methods are scarce.

From now on, and without loss of generality, we assume that the MOSP graph is connected.

Consider the graph of Fig. 4, where Subgraph<sub>1</sub> and Subgraph<sub>2</sub> are arbitrary and there is a single arc *Z* connecting them. We have that:

**Proposition 4** There always exists an optimal sequence of arcs to MOSP where all arcs of Subgraph<sub>1</sub> appear one after another without having any arcs of Subgraph<sub>2</sub> in between any two arcs of Subgraph<sub>1</sub> in the sequence and vice-versa, that is, all arcs of Subgraph<sub>2</sub> appear one after another without having any arc of Subgraph<sub>1</sub> in between any two arcs of Subgraph<sub>2</sub> in the sequence. (For proof see the Appendix)

Therefore, in order to find an optimal solution to an instance of the MOSP with similar characteristics as the graph problem schematized in Fig. 4, it is sufficient to search for solutions where all arcs of Subgraph<sub>1</sub> appear one after another without having any arcs of Subgraph<sub>2</sub> in between any two arcs of Subgraph<sub>1</sub> in the sequence (or vice versa, that is, all arcs of Subgraph<sub>2</sub> appear one after another in this optimal sequence without having any arc of Subgraph<sub>1</sub> in between any two arcs of Subgraph<sub>2</sub> in the sequence).

Using P4 recursively one can conclude that for a MOSP graph which is a ‘star’, as in Fig. 5, there is always an optimal sequence where all arcs of Subgraph<sub>*i*</sub> (any *i*) appear one after another without having arcs of (any of) the others subgraphs in between any two arcs of Subgraph<sub>*i*</sub> in the sequence.

### The refined branch-and-bound algorithm

The results presented in Section 2 are useful in reducing the search space only when the MOSP graph presents a structure similar to the one depicted in Fig. 5. The identification of a (vertex) separator in a graph is in general not an easy task (see, for instance, Ellis, Sudborough and Turner, 1994). However, for the special case of a star graph, it is sufficient to consider the removal of each vertex of the MOSP graph, one at a time, and verify whether the graph separates into a disconnected set of vertices. Whenever any separation is observed, the partitioned graphs can be solved independently. The branch-and-bound method of Yanasse (1997b) can be used to determine, independently, the optimal solutions for each one of the subgraphs obtained and a solution to the overall problem can be constructed based on these partial solutions.

Observe that some of the subgraphs obtained can be polynomially solvable cases of MOSP (see, for instance, Ellis, Sudborough and Turner, 1994; Yanasse, 1996, 1997a; Yanasse, Becceneri and Soma, 1998). A possible difficulty would be to identify which class the graph belongs. All the proposed algorithms assume beforehand that the graph being considered presents the conditions for which they were developed. We suggest the identification of the simple cases only, like trees and cycles, since these types of graphs can be checked quite easily.

We show next how to construct an optimal solution when a structure such as a star graph arises.

Let  $G(V, A)$  be a ‘star’ graph as in Fig. 5. Let us delete node  $k$ , the center of the star graph, and all arcs incident to it, subdividing  $G(V, A)$  into  $m$  disconnected subgraphs, Subgraph<sub>1</sub>, Subgraph<sub>2</sub>, . . . , Subgraph<sub>*m*–1</sub>, Subgraph<sub>*m*</sub>. Let  $PW_1, PW_2, \dots, PW_m$  be optimal solutions of the MOSPs corresponding to subgraphs Subgraph<sub>1</sub>, Subgraph<sub>2</sub>, . . . , Subgraph<sub>*m*–1</sub>, Subgraph<sub>*m*</sub>, respectively.

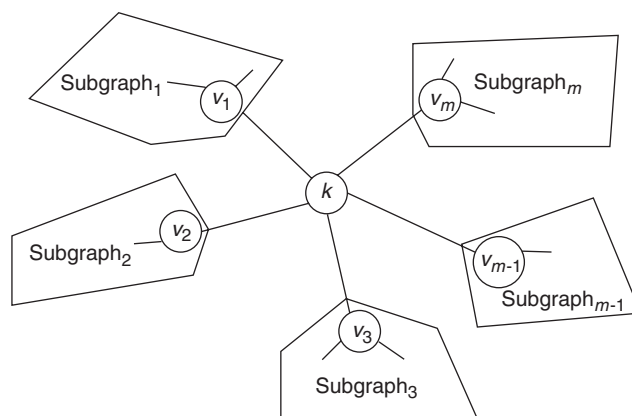


Fig. 5. ‘Star’ graph—Partition of a connected graph in  $m$  subgraphs linked by a single arc.

Let  $pw_1, pw_2, \dots, pw_m$  be their associated optimal solution values and assume, without loss of generality, that  $pw_1 \geq pw_m \geq pw_2 \geq pw_i, i = 3, \dots, m-1$ .

Let  $E\text{Subgraph}_i$  be the subgraph composed of  $\text{Subgraph}_i$  together with node  $k$  and the arc  $(v_i, k)$  that links node  $k$  to  $\text{Subgraph}_i$ . Let  $FPW_1, FPW_2, \dots, FPW_m$  be optimal solutions of the MOSPs corresponding to subgraphs  $E\text{Subgraph}_1, E\text{Subgraph}_2, \dots, E\text{Subgraph}_{m-1}, E\text{Subgraph}_m$ , respectively, with the restriction that vertex  $k$  is the first vertex open. Let  $fpw_1, fpw_2, \dots, fpw_m$  be their associated optimal solution values and assume, without loss of generality, that  $fpw_1 \geq fpw_m \geq fpw_2 \geq fpw_i, i = 3, \dots, m-1$ .

**Proposition 5** An optimal solution to the MOSP corresponding to  $G(V, A)$  can be obtained with either one of the following sequences:  $revFPW_1PW_j[(v_j, k)] \dots PW_s[(v_s, k)]FPW_m$  or  $revFPW_mPW_j[(v_j, k)] \dots PW_s[(v_s, k)]FPW_1$  where  $PW_i[(v_i, k)]$  denotes the sequence  $PW_i$  with arc  $(v_i, k)$  inserted just after the first time node  $v_i$  is opened and  $revFPW_i$  is sequence  $FPW_i$  in reverse order. (For a proof, see Appendix.)

Observe that the optimal solution of the subgraphs,  $\text{Subgraph}_1, \text{Subgraph}_2, \dots, \text{Subgraph}_{m-1}, \text{Subgraph}_m$  considered in  $P_5$  can be obtained independently, therefore, the number of enumerated nodes in a branch-and-bound scheme is reduced. Also (from the proof of  $P_5$  there is no need to solve every single subgraph to optimality. If at some stage a solution (not necessarily optimal) obtained for a subgraph is strictly smaller than any optimal value already obtained for another subgraph, or it is equal to the optimal value of at least three other subgraphs, then this solution can be used to build an optimal solution to the problem since its contribution will not influence the optimal overall value. Therefore, there is no need to keep on searching for the optimal solution of that subgraph. This may reduce the computational time substantially since we can interrupt the search for an optimal solution as soon as a convenient solution is generated.

In the branch-and-bound method of Yanasse (1997b) an optimal order in which the panel types are to be completed is sought. Starting from a root node,  $M$  (the total number of panel types) new nodes are generated, each one corresponding to the completion of a panel type. From each one of these nodes, a new branching is made, considering all the possibilities of completion of a panel type among the ones not yet completed. This is done repeatedly until all possible alternatives are analyzed. A greedy criterion is used to select the next node to be branched; the node selected is the one that currently produces the least number of open stacks. A depth search strategy is also used in order to quickly obtain a good feasible solution after a few branchings.

In  $P_4$  and  $P_5$ , a solution to MOSP was an arc traversing sequence. We use the information obtained from the MOSP problem as a graph traversing problem to improve the enumeration made of the panel types completion order. Recall that any arc traverse sequencing defines an order in which the panel types are completed. Different arc traversing sequences may lead to the same order in which the panel types are completed and it is this latter sequence that is relevant for it is the panel types completion order that defines the resulting number of open stacks. The necessary adjustments to ‘transform’ one solution to the other are straightforward.

Taking into account part of the ideas suggested previously, we have implemented an improved branch-and-bound method that gives only special consideration to the subgraphs that are trees. Algorithms for trees have already been suggested in the literature (see, for instance, Ellis, Sudborough and Turner, 1994) but we have implemented a modified version of the algorithm presented in Yanasse (1997a) which has the advantage of identifying automatically parts of the

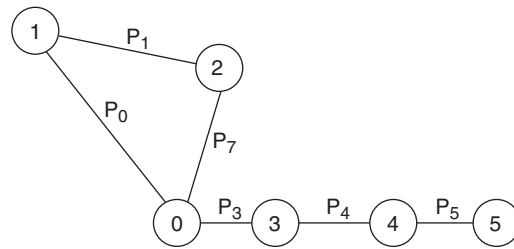


Fig. 6. Graph corresponding to a given instance.

MOSP graph that are trees. For the other non-tree parts of the graph, the algorithm implemented uses the regular branch-and-bound scheme presented previously with adjustments.

In the implemented algorithm, henceforth denoted **i-b-and-b**, we initially construct partial sequences of arcs that are the solutions  $PW$  and  $FPW$  of subgraphs of the MOSP graph that are trees. We start with nodes with degree 1 and we delete these nodes with degree 1 and the arcs incidental to them. The remaining nodes, if any, will have degree 0, 1, 2 or larger. The cases of 0 and 1 indicate that those nodes are still part of subgraphs that are trees and we repeat the process. As we keep on deleting these nodes with degree 1, a partial (optimal) sequence of the arcs incident to them is conveniently constructed.

After some iterations we are done (no nodes are left, therefore the graph was a tree) or there is a set of nodes left that forms a graph where all nodes have degree greater or equal to 2.

At this stage, there will be sets ( $PWs$ ,  $FPWs$ ) that are the solutions of the subgraphs that are trees, which are attached to some of the nodes, left on the remaining graph. For the enumeration phase, we propose to ‘represent’ each of these subgraphs by a special node, which is attached to the corresponding linking node in the remaining graph. The regular branch-and-bound scheme is applied to this extended graph (the remaining graph with the special nodes) where particular care is taken with the special nodes since they represent components of the original graph.

When considering these nodes in the enumeration process we know that in their place there are subgraphs (trees) whose optimal solutions have already been determined. Observe that the enumeration can be done this way and the solution obtained is guaranteed to be optimal because of  $P_4$ .

Consider the MOSP graph of Fig. 6. In this instance, the component of the graph containing nodes 3, 4 and 5 is a tree. In the initial steps of *algorithm i-b-and-b* partial sequences are calculated starting from nodes with degree 1, which is node 5.  $PW$  and  $FPW$  are constructed until a node with degree 3 or greater is reached (node 0 in this case).

After deleting all labeled nodes (3, 4, and 5) and all arcs incident to them, there will be no nodes of degree 0 or 1 left. Unlabeled nodes 0, 1, and 2 that are left have all degrees equal to 2. We attach special node 3 to node 0. In Fig. 7 we show the remaining graph with the special node 3 attached. Nodes 0, 1, 2 are, say, of type 0 (regular nodes) and, node 3, say, of type 1 (special node). The enumeration is done with this graph with some care when considering nodes type 1 in the branching since they represent components of the original graph (a subgraph having optimal sequences ( $PW$ ,  $FPW$ )).

In the next section we present some limited computational results examining the performance of the implemented algorithm.

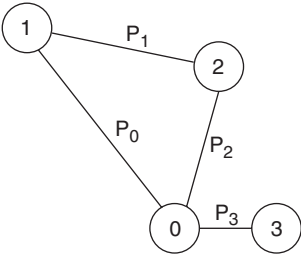


Fig. 7. Remaining graph with special nodes attached used in the enumeration.

Table 4  
Comparison of the algorithms using trees.

| <i>n</i> -panels | <i>n</i> -patterns | i-b-and-b | b-and-b |          |          |
|------------------|--------------------|-----------|---------|----------|----------|
|                  |                    |           | Min     | Max      | Average  |
| 10               | 9                  | 0         | 54      | 255      | 123.9    |
| 15               | 14                 | 0         | 119     | 8955     | 1336.7   |
| 20               | 19                 | 0         | 559     | 9135     | 1823.0   |
| 30               | 29                 | 0         | 1819    | 56,951   | 9061.8   |
| 40               | 39                 | 0         | 4248    | 710,781* | 14,265.3 |

obs. \*one of the instances did not reach optimality after 1 hour of CPU time.

Table 5  
Comparison of the algorithms for instances where there are some parts that are trees.

| <i>n</i> -panels | <i>n</i> -patterns | i-b-and-b |         |          | b-and-b      |            |            |
|------------------|--------------------|-----------|---------|----------|--------------|------------|------------|
|                  |                    | Min       | Max     | Average  | Min          | Max        | Average    |
| 14               | 10                 | 33        | 36      | 33.3     | 102          | 1,378      | 356        |
| 20               | 15                 | 63        | 319     | 168.2    | 506          | 1,305,785  | 252,593.6  |
| 22               | 14                 | 565       | 8365    | 2055.4   | 1.36 million | 92 million | 60 million |
| 30               | 15                 | 1117      | 248,511 | 37,796.6 | *            | *          | *          |

obs. \*none of the instances reached optimality after 1 hour of CPU time.

Computational results

The algorithm was implemented in C<sup>++</sup> (Borland C<sup>++</sup> 5.0) in a microcomputer Pentium MMX 233, with 64MB of RAM memory.

The data (a matrix of 0 and 1s) used to test the algorithm were generated randomly. The matrices generated were tested for equal rows and/or zero columns and these matrices were discarded when such cases were detected. A random tree generator was also implemented for the test.

The results of the tests performed are presented in Tables 4 and 5. In these tables we denote *i-b-and-b* the improved algorithm; *b-and-b* the previous existing algorithm suggested in Yanasse

(1997b);  $n_{\text{patterns}}$  is the total number of patterns in the instance and  $n_{\text{panels}}$  is the total number of panels in the instance. The algorithms are compared in terms of the number of nodes opened by the enumeration schemes for the computational time is directly proportional to this number. Different problems sizes were considered in the tests. For each problem size, ten instances were used.

In Table 4 we present the results with the tests made with trees. The results obtained indicate a clear superiority of algorithm *i-b-and-b* compared with algorithm *b-and-b*. This was expected since for trees, algorithm *i-b-and-b* is polynomial.

In Table 5 we present the results for the mixed case (there are some parts of the graph that are trees).

The results obtained show that the number of nodes generated using algorithm *i-b-and-b* is much smaller than the number of nodes generated using algorithm *b-and-b*. This was expected since the economy of nodes generated are due to the components of the graph that are trees.

For the cases where no special structure (no tree parts) is observed in the graph associated to the problem, both algorithms performed exactly the same, as expected.

According to our observations, one can closely estimate the observed runtime of the algorithm using the following proportion: 25,000 nodes generated per second.

## Summary and conclusions

Refinements on a branch-and-bound scheme were proposed which may produce major reductions in computational times when the MOSP graph has a ‘star’ structure. A question that may arise is whether such refinements are worthy. MOSP instances whose corresponding graphs present special structures can actually arise in real problems (see, for instance, the PLA used in the 8085 processor as indicated in Ferreira and Song (1992)); therefore, our concern for these special cases is genuine.

Using part of the proposed suggestions, an algorithm was implemented which does not require any prior information about the structure of the graph of the instance being solved. The implemented algorithm solves the problem in polynomial time if the corresponding MOSP graph is a tree; if parts of the MOSP graph are trees, these parts are solved in polynomial time and the remaining non-tree parts are solved using a regular enumeration scheme. All the general algorithms proposed previously in the literature, to the best of our knowledge, perform the same enumeration routines, even if the MOSP graph has special structures (for instance, a tree), hence, they take a much longer time to produce a solution. Also, it is worthy observing that the specialized algorithms simply do not work unless it is known beforehand that the instance being solved satisfies the particular conditions for which the algorithm was developed.

It may be of interest to know how the computational performance of a commercial software solver would be in solving MOSP. Some limited computational times using the CPLEX optimizer are shown in Table 6.

Although the number of instances used is small and the tests performed were made on different machines one can clearly notice the advantage of the proposed approach over CPLEX. The mathematical model used was the following:

Table 6

Comparison of the proposed approach and CPLEX.

| $n$ -panels | $n$ -patterns | i-b-and-b<br>Seconds* | CPLEX<br>Seconds** |
|-------------|---------------|-----------------------|--------------------|
| 6           | 6             | 0.01                  | 1.25               |
| 7           | 7             | 0.01                  | 7649.58            |
| 8           | 8             | 0.05                  | 18,384.55          |
| 9           | 9             | 0.06                  | 19,439.66          |
| 9           | 9             | 0.16                  | 32,165.87          |

\*Pentium MMX 233, with 64MB of RAM memory. \*\*SUN ULTRA 1, with 256 MB of RAM memory.

Minimize  $C$ 

$$\begin{aligned}
\text{Subject to } & \sum_{n=1}^{N=1} eP_n = M - C \\
& P_n \geq W_{n+1} - W_n \quad n = 1, \dots, N-1 \\
& P_n \geq 0 \quad n = 1, \dots, -1 \\
& eW_n \leq C \quad n = 1, \dots, N-1 \\
& x_{jn} A_j \leq W_n \quad j = 1, \dots, N; n = 1, \dots, N \\
& \sum_{n=1}^N x_{jn} = 1 \quad j = 1, \dots, N \\
& \sum_{j=1}^N x_{jn} = 1 \quad n = 1, \dots, N \\
& x_{jn} \in \{0, 1\} \quad j = 1, \dots, N; n = 1, \dots, N \\
& W_n \quad n = 1, \dots, N \\
& \text{is a vector of 0 and 1s}
\end{aligned}$$

where

 $N$  is the number of patterns to be sequenced; $M$  is the total number of panel types; $A_j$  is an  $M$ -vector of 1 and 0s giving the panel types in pattern  $j$  (1 if panel type is present, 0 otherwise);time  $n$  is the moment in time after processing the  $n$ th pattern, but before any stack is switched; $C$  is the maximum number of open stacks; $e$  is the 1XM vector of 1s; $x_{jn} = 1$  if pattern  $j$  is the  $n$ th pattern in the sequence,  
0 otherwise; $W_n$  is an  $M$ -vector of 1 and 0s giving the panel stacks (open and the ones that are just closed with the  $n$ th pattern processed) at time  $n$ ;

$P_n$  is an  $M$ -vector that shows the panel types switches (new panel type stacks in place of completed stacks) that occur at instant  $n$ .

This formulation of MOSP was proposed in Yanasse (1997b), where the interested reader can also find a detailed explanation.

Another concern that may eventually arise is whether given an instance of MOSP with  $N$  patterns and  $M$  panels (specially when  $M$  is greater than  $N$ ), is it better to perform a search enumerating the permutations of the panel types' completion order (as proposed) or is it better to enumerate all possible pattern sequencing permutations (as in Faggioli and Bentivoglio, 1998)? It can be shown that a careful enumeration of the completion of panel types is never worse than enumerating pattern permutations (see Yanasse, 1998).

Further improvements on the enumeration scheme, for example, establishing priorities to branch the nodes, are currently being investigated.

There are many other problems similar to MOSP and the idea of extending the approach used in this work to these problems can be attempted. We have not pursued this idea further but it is certainly a possibility for future studies.

## Acknowledgements

This work was partially funded by CNPq, a Brazilian Government Agency for Science and Technology Development, and by FAPESP. We would like to thank Dr J. C. Becceneri for providing the computational results using CPLEX. We thank the anonymous referees for their valuable comments.

## References

- Ellis, J.A., Sudborough, I.H., Turner, J.S., 1994. The vertex separation and search number of a graph. *Information and Computation* 113, 50–79.
- Faggioli, E., Bentivoglio, C.A., 1998. Heuristic and exact methods for the cutting sequencing problem. *European Journal of Operational Research* 110, 564–575.
- Ferreira, A.G., Song, S.W., 1992. Achieving optimality for gate matrix layout and PLA folding: a graph theoretic approach. *Integration, The VLSI Journal* 14(2), 173–195.
- Garey, M.R., Graham, R.L., Johnson, D.S., Knuth, D.E., 1978. Complexity results for bandwidth minimization. *SIAM Journal of Applied Mathematics* 34, 477–495.
- Kashiwabara, T., Fujisawa, T., 1979. NP-Completeness of the problem of finding a minimum clique number interval graph containing a given graph as a subgraph. *Proceedings 1979 IEEE International Symposium on Circuits and Systems*, Tokyo, Japan, pp. 657–660.
- Kinnersley, N.G., 1992. The vertex separation number of a graph equals its path width. *Information Processing Letters* 4, 345–350.
- Linhares, A., Yanasse, H.H., 2002. Connections between cutting-pattern sequencing, VLSI design, and flexible machines. *Computers and Operations Research* 29(12), 1759–1772.
- Lins, S., 1989. Traversing trees and scheduling tasks for duplex corrugator machines. *Pesquisa Operacional* 9(1), 40–54.
- Madsen, O.B.G., 1988. An application of travelling-salesman routines to solve pattern-allocation problems in the glass industry. *Journal of the Operational Research Society* 39(3), 249–256.
- Möhring, R.H., 1990. Graph problems related to gate matrix layout and PLA folding. *Computing* 7, 17–51.

- Ohtsuki, T., Mori, H., Kuh, E.S., Kashiwabara, T., Fujisawa, T., 1979. One-dimensional logic gate assignment and interval graphs. *IEEE Transactions on Circuits and Systems* 26, 675–684.
- Tang, C.S., Denardo, E.V., 1988. Models arising from a flexible manufacturing machine, Part I: minimization of the number of tool switches. *Operations Research* 36, 767–777.
- Wing, O., Huang, S., Wang, R., 1985. Gate matrix layout. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 4, 220–231.
- Yanasse, H.H., 1996. Minimization of open orders—polynomial algorithms for some special cases. *Pesquisa Operacional* 16(1), 1–26.
- Yanasse, H.H., 1997a. An exact algorithm for the tree case of the minimization of open orders problem, Technical Report LAC-001/97, INPE, São José dos Campos, SP, Brazil.
- Yanasse, H.H., 1997b. On a pattern sequencing problem to minimize the maximum number of open orders. *European Journal of Operational Research* 100, 454–463.
- Yanasse, H.H., 1997c. A transformation for solving a pattern sequencing problem in the wood cut industry. *Pesquisa Operacional* 17(1), 57–70.
- Yanasse, H.H., 1998. A note on generating solutions of a pattern sequencing problem to minimize the maximum number of open orders, Technical Report LAC-002/98, INPE, São José dos Campos, SP, Brazil.
- Yanasse, H.H., Becceneri, J.C., Soma, N.Y., 1998. An exact algorithm for a special case of a pattern sequencing problem. Presented at the XXX SBPO, held in Curitiba, PR, Brazil.
- Yuen, B.J., 1991. Heuristics for sequencing cutting patterns. *European Journal of Operational Research* 55(2), 183–190.

## Appendix—Proof of Propositions 4 and 5

### Proof of proposition 4.

Let  $P_1, P_2, \dots, P_n$  and  $Q_1, Q_2, \dots, Q_m$  be the arcs in Subgraph<sub>1</sub> and Subgraph<sub>2</sub>, respectively. Without loss of generality, assume that arc  $Z$  is part of Subgraph<sub>1</sub>, that is, it is one of the  $P_i$ s in the set  $\{P_1, P_2, \dots, P_n\}$ . Let Seq<sub>ot</sub> =  $P_l Q_j Q_s P_k \dots Q_z P_i P_x Q_v \dots P_n \dots Q_m \dots Q_t P_r Q_r \dots Q_x$ , be an optimal sequence of the MOSP associated with the graph of Fig. 4. Let Seq<sub>1</sub> equal the sequence  $P_1 P_k \dots P_i P_x \dots P_n \dots P_l$  obtained from Seq<sub>ot</sub> after deleting all arcs of Subgraph<sub>2</sub> and Seq<sub>2</sub> equal the sequence  $Q_j Q_s \dots Q_z Q_v \dots Q_n \dots Q_t Q_r \dots Q_x$  obtained from Seq<sub>ot</sub> after deleting all arcs of Subgraph<sub>1</sub>. Assume that the maximum number of open nodes in Subgraph<sub>1</sub>, following Seq<sub>1</sub>, occurs just after arc  $P_i$  is sequenced. Let Seq<sub>ot\_pl</sub> equal to  $P_l Q_j Q_s P_k \dots Q_z P_i$ , that is, the subsequence obtained from Seq<sub>ot</sub> deleting all arcs in the sequence after  $P_i$  and Seq<sub>ot\_p2</sub> equal to  $P_x Q_v \dots P_n \dots Q_n \dots Q_t P_i Q_r \dots Q_x$ , that is, the subsequence obtained from Seq<sub>ot</sub> deleting all arcs prior to and including  $P_i$ . Let Seq<sub>2\_pl</sub> equal to  $Q_j Q_s \dots Q_z$ , that is, the subsequence obtained from Seq<sub>ot\_pl</sub> deleting all arcs of Subgraph<sub>1</sub> and Seq<sub>2\_p2</sub> equal to  $Q_v \dots Q_m \dots Q_t Q_r \dots Q_x$ , that is, the subsequence obtained from Seq<sub>ot\_p2</sub> deleting all arcs of Subgraph<sub>1</sub>. Let  $v(seq)$  be the maximum resulting number of open nodes following sequence  $seq$ . Using P2 we can show that  $v(Seq_{ot\_pl}) \geq v(Seq_{2\_pl} Seq_1)$  and, afterwards, that  $v(Seq_{ot}) \geq v(Seq_{2\_pl} Seq_1 Seq_{2\_p2})$ . Therefore, Seq<sub>2\_pl</sub> Seq<sub>1</sub> Seq<sub>2\_p2</sub> is also optimal. Following a similar reasoning, we can show that  $v(Seq_1) \leq v(Seq_{2\_pl} Seq_1)$  and afterwards that  $v(Seq_1 Seq_2) \leq v(Seq_{2\_pl} Seq_1 Seq_{2\_p2})$ . Since Seq<sub>2\_pl</sub> Seq<sub>1</sub> Seq<sub>2\_p2</sub> is optimal, so is Seq<sub>1</sub> Seq<sub>2</sub> (or revSeq<sub>2</sub> revSeq<sub>1</sub>, by P1).

### Proof of P5.

Because of the special topology of  $G(V, A)$ , following any possible sequence of the arcs, the first arc sequenced opens at least one of the vertices belonging to one of the subgraphs Subgraph<sub>i</sub>,

$i = 1, 2, \dots, m$ , and the last arc sequenced will also complete a vertex belonging to one of the subgraphs  $\text{Subgraph}_i$ ,  $i = 1, 2, \dots, m$ . Without loss of generality, suppose the sequence starts with an arc incident to a node in  $\text{Subgraph}_1$  and finishes with an arc incident to a node in  $\text{Subgraph}_m$ .

Since the graph is connected, whatever the sequence we choose, when *any* arc of the second, third,  $\dots$ ,  $(m-1)^{\text{th}}$  subgraph is sequenced, there will be at least one open node, in some other subgraph or node  $k$  itself. Therefore the best we can do for any possible sequence of the arcs in  $\text{Subgraph}_i$ ,  $i = 2, 3, \dots, m-1$ , when we start with an arc in  $\text{Subgraph}_1$  and we finish with an arc in  $\text{Subgraph}_m$  is  $pw_i + 1$ . For the arcs of the first ( $\text{Subgraph}_1$ ) and the last ( $\text{Subgraph}_m$ ) subgraphs in the sequence, we cannot ensure anything. Therefore, the best we can do for any sequence that starts with an arc in  $\text{Subgraph}_1$  and ends with an arc in  $\text{Subgraph}_m$  is  $\text{LB} = \{pw_1, pw_2 + 1, \dots, pw_{m-2} + 1, pw_{m-1} + 1, pw_m\}$ . On the other hand, it is always possible to find a sequence starting with an arc in  $\text{Subgraph}_1$  and ending with an arc in  $\text{Subgraph}_m$  whose solution value is bounded by  $UP = \max\{pw_1 + 1, pw_2 + 1, \dots, pw_{m-2} + 1, pw_{m-1} + 1, pw_m + 1\}$ , by just using the following sequence:  $PW_1[(v_1, k)] PW_2[(v_2, k)] \dots (v_m, k) PW_m$ , where  $PW_i[(v_i, k)]$  denotes the sequence  $PW_i$  with arc  $(v_i, k)$  inserted just after the first time node  $v_i$  is opened. Observe that after  $PW_1[(v_1, k)]$ , the only node open is  $k$ , and the maximum number of open orders up to this point is  $pw_1$  or  $pw_1 + 1$ ; for the subsequent sequences,  $PW_i[(v_i, k)]$ ,  $i = 2, 3, \dots, m-1$ , the maximum number of open orders will be  $pw_i + 1$  where the additional 1 is due to the open node  $k$ . At the beginning of the last part of the sequence,  $(v_m, k) PW_m$ , node  $k$  is the single node opened at this point. Node  $k$  is closed when we sequence  $(v_m, k)$ . As we sequence  $PW_m$ , the worst we can do is  $pw_m + 1$ .

If the sequence starts with an arc incident to a node in  $\text{Subgraph}_1$  and finishes with an arc incident to a node in the same  $\text{Subgraph}_1$ , then, following a similar reasoning, the best we can do is  $\max\{pw_1, pw_2 + 1, \dots, pw_{m-2} + 1, pw_{m-1} + 1, pw_m + 1\}$ .

Observe, hence, that if  $pw_1 \geq pw_m \geq pw_2 \geq pw_i$ ,  $i = 3, \dots, m-1$ , then  $pw_2 + 1$  is a lower bound for the optimal solution value for the MOSP. Therefore, if  $pw_1 = pw_2$ , the solution  $PW_1[(v_1, k)] PW_2[(v_2, k)] \dots (v_m, k) PW_m$  with value  $UP$  is optimal. So, the cases that need attention are those when  $pw_1 \geq pw_m \geq pw_2$ . Assume, therefore, that  $pw_1 \geq pw_m \geq pw_2$ .

Let  $epw_i$  be the optimal solution value of a MOSP corresponding to  $\text{ESubgraph}_i$ . By P2,  $epw_i \geq pw_i$ . Also,  $epw_i \leq pw_i + 1$ , since the sequence  $PW_i[(v_i, k)]$  will always produce at most  $pw_i + 1$  open orders. If  $epw_1 = pw_1 + 1$ , then by P2, the optimal solution to the original graph problem cannot be better than  $epw_1 = pw_1 + 1$ , hence again, the solution  $PW_1[(v_1, k)] PW_2[(v_2, k)] \dots (v_m, k) PW_m$  we proposed previously with value  $UP = pw_1 + 1$  is optimal.

As can be seen, we cannot avoid the increase of at least +1 in the optimal values of the partial (optimal) sequences corresponding to subgraphs  $\text{Subgraph}_2, \dots, \text{Subgraph}_{m-2}, \text{Subgraph}_{m-1}$ . Assuming  $pw_1 \geq pw_m \geq pw_2 \geq pw_i$ ,  $i = 3, \dots, m-1$ , in order to minimize the maximum number of open orders, a strategy that always produces an optimal solution is to maintain the subsequences corresponding to  $\text{Subgraph}_1$  and  $\text{Subgraph}_m$  in the first and last place of the sequence (or vice versa), therefore, maintaining the chance of finding a sequence where the maximum number of open orders observed does not exceed  $pw_1$ . Any other position for the subsequence corresponding to  $\text{Subgraph}_1$  results in at least  $pw_1 + 1$  open orders, which is as good as the solution we proposed previously with value  $UP$  equal to  $pw_1 + 1$ . We will focus our attention on subgraphs  $\text{Subgraph}_1$  and  $\text{Subgraph}_m$ ; for the remaining others we know how to sequence them keeping the open orders limited to  $pw_1$  (and, hence, guaranteeing optimality of the solution).

Observe that if  $pw_1 > pw_m$ , then it is sufficient to focus on Subgraph<sub>1</sub> alone since the addition of +1 to the optimal solution value of the last part of the sequence (corresponding to Subgraph<sub>m</sub>) does not affect the maximum value of the whole sequence.

We also have that, for all  $i$ ,  $pw_i + 1 \geq fpw_i \geq epw_i \geq pw_i$  since  $revPW_i[(v_i, k)]$  produces at most  $pw_i + 1$  open orders even with the condition that vertex  $k$  is the first vertex open and  $epw_i$  is the optimal solution value for ESubgraph<sub>i</sub> without any additional constraint.

Observe that  $FPW_i$  or its reverse,  $revFPW_i$ , may not produce an optimal solution when we impose that vertex  $k$  is open first and remains open from that point onwards since the maximum number of open stacks may be greater than  $pw_i + 1$ .

Without loss of generality, we assume that  $fpw_1 \geq fpw_m \geq fpw_i$ ,  $i = 2, 3, \dots, m - 1$  (in addition to  $pw_1 \geq pw_m \geq pw_i$ ,  $i = 2, 3, \dots, m - 1$ ).

If the arcs belonging to Subgraph<sub>1</sub> are placed at the beginning, vertex  $v_1$ , once opened, remains open from that point onwards until  $(v_1, k)$  is sequenced. When  $(v_1, k)$  is sequenced, node  $k$  will be opened and will remain open while the remaining arcs belonging to Subgraph<sub>1</sub> are sequenced. Therefore, the best we can do when the subsequence of all the arcs belonging to Subgraph<sub>1</sub> is placed at the beginning of the sequence is to follow the subsequence  $revFPW_1$ .

When we start to sequence the arcs belonging to Subgraph<sub>m</sub> at the end of the sequence, node  $k$  or node  $v_m$  will be opened. If node  $k$  is open and  $(v_m, k)$  is sequenced to close node  $k$ , then node  $v_m$  is opened. Therefore, the best we can do when the subsequence of the arcs of Subgraph<sub>m</sub> are placed at the end of the sequence is to follow the subsequence  $FPW_m$ .

So, the sequence  $revFPW_1PW_j[(v_j, k)] \dots PW_s[(v_s, k)]FPW_m$  is the best we can do when we leave the subsequences corresponding to Subgraph<sub>1</sub> and Subgraph<sub>m</sub> in the first and last place, respectively, of the sequence. The best we can do when we leave the subsequences corresponding to Subgraph<sub>1</sub> and Subgraph<sub>m</sub> in the last and first place, respectively, is  $FPW_mPW_j[(v_j, k)] \dots PW_s[(v_s, k)]revFPW_1$  which produces the same maximum number of open stacks. Both sequences are, therefore, optimal for the MOSP corresponding to this ‘star’ graph. The optimal solution value is equal to  $\max \{fpw_1, fpw_m, pw_2 + 1\}$  which may be smaller or equal to  $pw_1 + 1$ .